

Task 1: Process Scheduling Algorithms

Round Robin (RR)

- Each process gets a fixed **Time Quantum (TQ)**.
- Processes are executed in **circular order**.
- If a process does not finish during its quantum, it goes to the **end of the Ready Queue**.
- Good for **time-sharing systems**.
- Fair because every process gets CPU time.

Example: 5 Processes

Assume all arrive at time 0 and **Time Quantum = 2**:

Process Burst Time

P1	5
P2	3
P3	1
P4	4
P5	2

Gantt Chart:

0 2 4 5 7 9 11 12 13 15
| P1 | P2 | P3 | P4 | P5 | P1 | P2 | P4 | P1 |

Scheduling Comparison

Algorithm	Main Idea	Advantage	Disadvantage
FCFS	First process arrives, runs first	Simple	Convoy effect, poor response
SJF	Shortest job first	Low average waiting time	Long jobs may starve
Priority	Highest priority runs first	Important tasks run first	Starvation possible
Round Robin	Each process gets a time quantum	Fair, good response	Performance depends on quantum
Multilevel Queue	Separate queues for different process types	Flexible	Complex, possible starvation

Task 2: Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	✗
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	✓**
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	✓**

* Average recursion stack.

** When implemented with a stable internal sorting step.

Quick Use Cases

- **Bubble:** Educational purposes, very small datasets.
- **Selection:** Small datasets, when memory is limited.
- **Insertion:** Small or nearly sorted data.
- **Merge:** Large datasets, linked lists, external sorting.
- **Quick:** General-purpose fast sorting.
- **Heap:** When guaranteed $O(n \log n)$ and low extra memory are needed.
- **Counting:** Integers with a limited range.
- **Radix:** Large numbers or strings with fixed-length keys.

Easy Rule to Remember

Simple sorts: Bubble, Selection, Insertion → usually $O(n^2)$

Efficient comparison sorts: Merge, Quick, Heap → around $O(n \log n)$

Non-comparison sorts: Counting, Radix → can be faster than $O(n \log n)$ under specific conditions.

Task 3: Searching Algorithms

Algorithm	Average	Worst	Space	Requirement
Linear Search	$O(n)$	$O(n)$	$O(1)$	No sorting required
Binary Search	$O(\log n)$	$O(\log n)$	$O(1)$	Sorted data

Algorithm	Average	Worst	Space	Requirement
Jump Search	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(1)$	Sorted data
Interpolation Search	$O(\log \log n)$	$O(n)$	$O(1)$	Sorted + uniformly distributed data
Exponential Search	$O(\log n)$	$O(\log n)$	$O(1)$	Sorted data
Hash Search	$O(1)$ average	$O(n)$	$O(n)$	Hash table

Key Points

Linear Search

- Checks elements one by one.
- Works on sorted or unsorted data.
- Simple but slow for large datasets.

Binary Search

- Checks the middle element and eliminates half of the search space.
- Very efficient, but requires sorted data.

Jump Search

- Jumps through blocks instead of checking every element.
- Requires sorted data.

Interpolation Search

- Estimates where the value should be.
- Very fast for uniformly distributed numbers.
- Can become $O(n)$ with poor distribution.

Exponential Search

- Quickly finds a possible range, then uses binary search.
- Useful when the size of the sorted dataset is unknown.

Hash-Based Search

- Uses a hash function to locate data.
 - Average lookup is **$O(1)$** .
 - Used in dictionaries, maps, and hash tables.
-

Task 4: Collections & Data Structures in Python, Java, and C++

Array

Language	Implementation
Python	Lists are dynamic arrays
Java	Arrays have fixed size and store primitive values or references
C++	Built-in arrays have fixed size; <code>vector</code> is dynamic

- Fast indexing: **O(1)**
- Good for sequential data.
- Insertion/deletion in the middle is usually **O(n)**.

List

Python: `list`

- Dynamic array.
- Fast indexing.
- Usually O(1) append.

Java: `ArrayList` / `LinkedList`

- `ArrayList`: dynamic array.
- `LinkedList`: linked nodes.

C++: `vector` / `list`

- `vector`: dynamic array.
- `list`: doubly linked list.

Set

- Stores **unique elements**.
- Python `set`, Java `HashSet`, C++ `unordered_set` generally provide average **O(1)** lookup.
- Ordered sets use tree structures and usually provide **O(log n)**.

Map

Stores **key-value pairs**.

Language Common Map

Python `dict`

Java `HashMap, TreeMap`

C++ `unordered_map, map`

- Hash-based maps: average **O(1)** lookup.
- Tree-based maps: **O(log n)** lookup.
- Maps are useful for fast lookup using keys.

Task 5: Stack vs Heap Memory

Feature	Stack	Heap
Allocation	Automatic	Dynamic
Lifetime	Usually tied to function/scope	Until released/managed
Speed	Very fast	Generally slower
Size	Limited	Usually larger
Organization	LIFO	Flexible
Threads	Each thread has its own stack	Usually shared process memory
Common Use	Local variables, function calls	Dynamic objects/data

Stack

- Stores function calls and local variables.
- Automatically managed.
- Fast.
- Limited in size.

Heap

- Used for dynamically allocated memory.
- Objects can live longer than the function that created them.
- Larger and more flexible.
- Requires memory management, either manually or through a garbage collector.

Easy memory trick:

Stack = temporary + automatic

Heap = dynamic + flexible

Task 6: Ordered vs Unordered Collections in C++

Sets

Type	Order	Duplicates	Implementation	Operations
<code>set</code>	Sorted	✗	Red-Black Tree	$O(\log n)$
<code>unordered_set</code>	No order	✗	Hash Table	$O(1)$ average
<code>multiset</code>	Sorted	✓	Red-Black Tree	$O(\log n)$

Use Cases

- **set:** Need unique values **and sorted order**.
 - **unordered_set:** Need fast lookup and don't care about order.
 - **multiset:** Need sorted values **with duplicates**.
-

Maps

Type	Order	Duplicate Keys	Implementation	Operations
<code>map</code>	Sorted by key	✗	Red-Black Tree	$O(\log n)$
<code>unordered_map</code>	No order	✗	Hash Table	$O(1)$ average
<code>multimap</code>	Sorted by key	✓	Red-Black Tree	$O(\log n)$

Important Difference

`map`

→ sorted keys

→ unique keys

`unordered_map`

→ no sorting

→ unique keys

→ faster average lookup

`multimap`

→ sorted keys

→ duplicate keys allowed